

《优化理论与算法》第 12 次作业

- 作业截止于**周五 (6/5/2026)**，在 Canvas 系统中提交 PDF 或照片文件
- 作业题目的 tex 文件可以在 Canvas 中下载
- 鼓励相互讨论，但不要抄袭.

1 一阶算法

1.1 给定 $\lambda > 0, x \in \mathbb{R}^n$ ，考虑如下函数形式（岭回归问题）

$$\min f(x) = \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_2^2$$

- 最优解 x^* 的显示表达式是什么？
- 如果利用梯度下降法，其迭代算法框架是什么？
- 如果利用随机梯度下降法，其迭代算法框架是什么？
- 对比上述三种方法的优劣。

1.1 solution

给定 $\lambda > 0, x \in \mathbb{R}^n$ ，考虑如下函数形式（岭回归问题）

$$\min_x f(x) = \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_2^2$$

最优解 x^* 的显示表达式 对目标函数求梯度并令其为 0：

$$\nabla f(x) = A^T(Ax - b) + 2\lambda x = 0$$

整理可得：

$$(A^T A + 2\lambda I)x = A^T b$$

因此最优解为：

$$x^* = (A^T A + 2\lambda I)^{-1} A^T b$$

梯度下降法迭代框架 梯度下降法的迭代公式为：

$$x^{k+1} = x^k - \alpha_k \nabla f(x^k)$$

其中梯度：

$$\nabla f(x^k) = A^T(Ax^k - b) + 2\lambda x^k$$

迭代步骤：

1. 初始化 x^0 ，步长 α_k ；
2. 计算梯度 $g_k = A^T(Ax^k - b) + 2\lambda x^k$ ；
3. 更新 $x^{k+1} = x^k - \alpha_k g_k$ ；
4. 重复直到收敛（如 $\|g_k\| < \epsilon$ ）。

随机梯度下降法迭代框架 将目标函数写成样本形式：

$$f(x) = \frac{1}{2m} \sum_{i=1}^m (a_i^T x - b_i)^2 + \lambda \|x\|_2^2$$

随机选取样本 i_k ，用单样本梯度近似全梯度：

$$\nabla f_{i_k}(x^k) = (a_{i_k}^T x^k - b_{i_k}) a_{i_k} + 2\lambda x^k$$

迭代步骤：

1. 初始化 x^0 ，步长序列 α_k ；
2. 随机采样样本 i_k ；
3. 计算随机梯度 $g_k = (a_{i_k}^T x^k - b_{i_k}) a_{i_k} + 2\lambda x^k$ ；
4. 更新 $x^{k+1} = x^k - \alpha_k g_k$ ；
5. 重复直到收敛。

Table 1: 岭回归求解方法对比

方法	优点	缺点
闭式解	直接得到精确最优解	大规模矩阵求逆复杂度高 ($O(n^3)$)
梯度下降	实现简单，收敛稳定	全梯度计算需遍历所有样本，大规模数据效率低
SGD	单样本更新，适合超大规模/在线学习	梯度噪声大，收敛过程震荡，调参复杂

三种方法对比

1.2 总结这段时间学习的所有一阶方法。总结内容，包括方法适用场景，步长的选取方法，不同条件下的收敛性等，以及各个方法的优缺点。可以用文字结合表格展示。

2 牛顿法

2.1 其中初始点设为 $x_0 = 0$ ，利用牛顿法找到函数 $f(x) = \cos(3x) - \sin(x)$ 的根。

2.1 solution

牛顿法求根的迭代公式为

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

带入方程表达式有

$$x_{k+1} = x_k - \frac{\cos 3x_k - \sin x_k}{-3 \sin 3x_k - \cos x_k}$$

迭代 1: $x = 1.000000000000$, $f(x) = -1.83\text{e}+00$

迭代 2: $x = -0.9005240988$, $f(x) = -1.21\text{e}-01$

迭代 3: $x = -0.7161246123$, $f(x) = 1.10\text{e}-01$

迭代 4: $x = -0.7789271624$, $f(x) = 9.27\text{e}-03$

迭代 5: $x = -0.7853176603$, $f(x) = 1.14\text{e}-04$

迭代 6: $x = -0.7853981504$, $f(x) = 1.83\text{e}-08$

迭代 7: $x = -0.7853981634$, $f(x) = 5.55\text{e}-16$

迭代 8: $x = -0.7853981634$, $f(x) = -4.44\text{e}-16$

收敛到解

函数根的近似值为：

$$x = -0.7853981634$$

精确解是

$$x = -\frac{\pi}{4}$$

也可以计算演示。

3 阻尼牛顿法

3.1 《Convex Optimization》(Boyd and Vandenberghe) 教材课后题 9.9

3.1 solution

$$\lambda(x) = (\nabla f(x)^T \nabla^2 f(x)^{-1} \nabla f(x))^{\frac{1}{2}}$$

先证明第一个等式，有

$$\sup_{v^T \nabla^2 f(x) v = 1} (-v^T \nabla f(x)) \stackrel{u = \nabla^2 f(x)^{-\frac{1}{2}} v}{=} \sup_{\|u\|_2 = 1} (-u^T \nabla^2 f(x)^{-\frac{1}{2}} \nabla f(x))$$

$$u = -\frac{\nabla^2 f(x)^{-\frac{1}{2}} \nabla f(x)}{\left\| \nabla^2 f(x)^{-\frac{1}{2}} \nabla f(x) \right\|_2} = \left\| \nabla^2 f(x)^{-\frac{1}{2}} \nabla f(x) \right\|_2 = \lambda(x)$$

取最大值只需要 u 与 $\nabla^2 f(x)^{-\frac{1}{2}} \nabla f(x)$ 相反即可。

接下来证明第三个等式，有

$$\sup_{v \neq 0} \frac{-v^T \nabla f(x)}{(v^T \nabla^2 f(x) v)^{\frac{1}{2}}} \stackrel{\substack{u^T \nabla^2 f(x) u = 1, \\ v = \alpha u, \alpha > 0}}{=} \sup_{u^T \nabla^2 f(x) u = 1} \frac{-\alpha u^T \nabla f(x)}{\alpha} = \sup_{v^T \nabla^2 f(x) v = 1} -v^T \nabla f(x)$$

得证。

3.2 考虑最小化如下目标函数

$$f(x_1, x_2) = e^{x_1 + 2x_2 - 0.1} + e^{x_1 - 3x_2 - 0.1} + e^{-x_1 - 0.1}.$$

- 利用一阶最优性条件，找到最优解 x^* 与最优值 f^* .
- 考虑初始点 $x_0 = (5, 2)$. 使用阻尼牛顿法结合回溯线搜索 ($\alpha = 0.1$ and $\beta = 0.5$.) 进行迭代求解直到 $f(x_k) - f^* < 10^{-10}$.
绘图 $f(x_k) - f^*$ 与迭代次数 k 的关系
- 牛顿法计算每次迭代计算海森矩阵的逆矩阵耗时较多，在求解大规模问题中常用以下两种估计方法：

- 重复利用海森矩阵：每次 N 次迭代记录一次当前点的海森矩阵，然后重复利用该矩阵当做“真实”的海森矩阵进行迭代 N 次，之后更新海森矩阵，以此类推。
- 对角估计：将海森矩阵用其对角元进行估计。也就是，每次迭代仅求解 n 个二阶导数 $\frac{\partial^2 f(x)}{\partial x_i^2}$ ，利用该对角矩阵修正 $-\nabla f(x)$ 方向。用这个方法，可以减少运行时间和储存成本。

对上述两种改进阻尼牛顿法，对 (b) 中问题进行求解，将其求解结果与常规的阻尼牛顿法进行比较。

3.2 solution

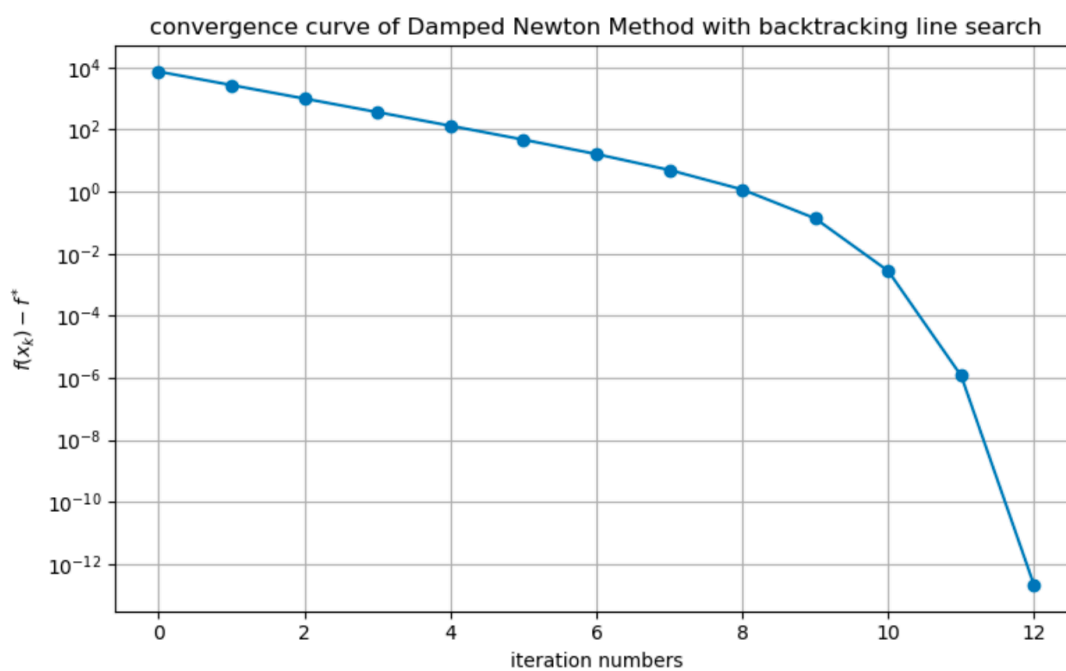
(a)

$$\nabla f(x) = \begin{pmatrix} e^{x_1+2x_2-0.1} + e^{x_1-3x_2-0.1} - e^{-x_1-0.1} \\ 2e^{x_1+2x_2-0.1} - 3e^{x_1-3x_2-0.1} \end{pmatrix} = 0$$

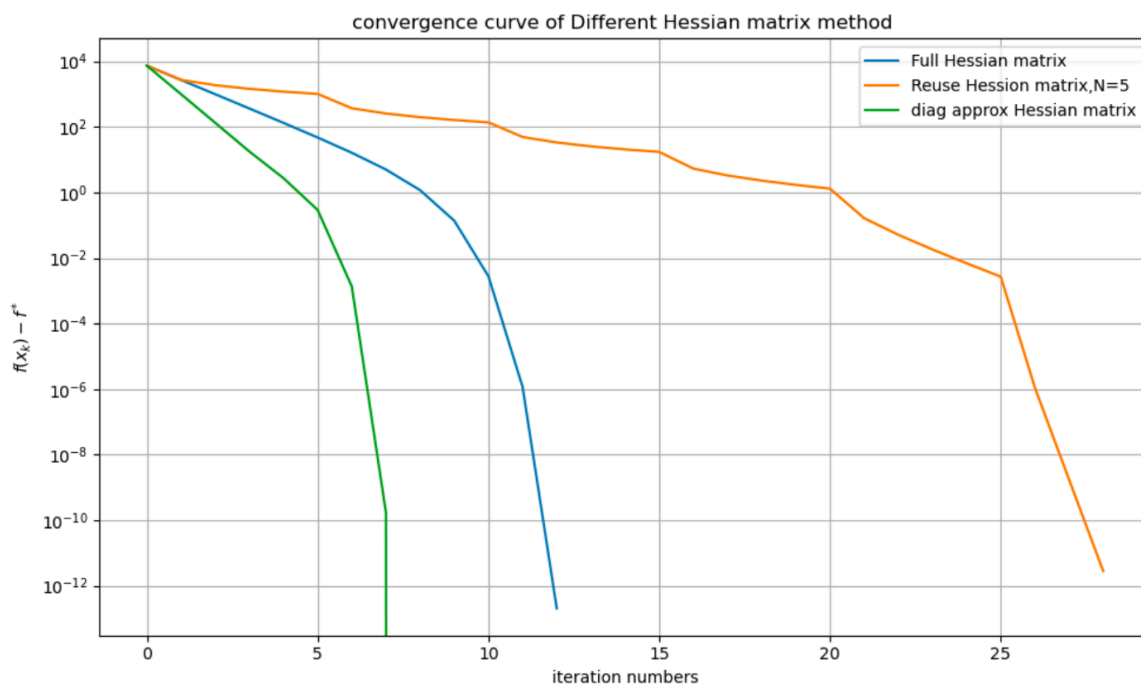
求解方程组得到

$$x^* = \left(\frac{3}{10} \ln \frac{3}{5} + \frac{2}{10} \ln \frac{2}{5}, \frac{1}{5} \ln \frac{3}{2} \right)^T, \quad f^* = 2e^{-\frac{3}{10} \ln \frac{3}{5} - \frac{2}{10} \ln \frac{2}{5} - 0.1}$$

(b)



(c)



4 拟牛顿法

4.1 线搜索是优化方法中的一个重要步骤。我们已经了解了基本的回溯线搜索 (**Armijo-Wolfe 线搜索**) 是一种重要的非精确线搜索。假设我们希望找到 t ，使得 $f(x_k + t d_k) \approx \min_t f(x_k + t d_k)$ ，其中 d_k 是第 k 步的搜索方向；在梯度法中， $d_k = -\nabla f(x_k)$ ，而在牛顿法中， $d_k = -\nabla^2 f(x_k)^{-1} \nabla f(x_k)$ 。

更进一步，为了描述这些线搜索条件，取参数 $0 < c_1 < c_2 < 1$ ，得到两个条件

$$\text{充分下降条件: } f(x_k + t_k d_k) \leq f(x_k) + c_1 t_k \nabla f(x_k)^T d_k$$

$$\text{曲率条件: } \nabla f(x_k + t_k d_k)^T d_k \geq c_2 \nabla f(x_k)^T d_k$$

(如果定义 $g(t) = f(x_k + t d_k)$ ，那么曲率条件就是 $g'(t_k) \geq c_2 g'(0)$)

这两个条件合在一起称为 (弱) Wolfe 条件。可以证明，只要 d 是一个下降方向，集合 $\{t \mid t \text{ 满足 Wolfe 条件}\}$ 就有非空的内部。[《数值优化》(Nocedal 和 Wright)，第 3 章]

该线搜索可以利用如下二分法求解：

```

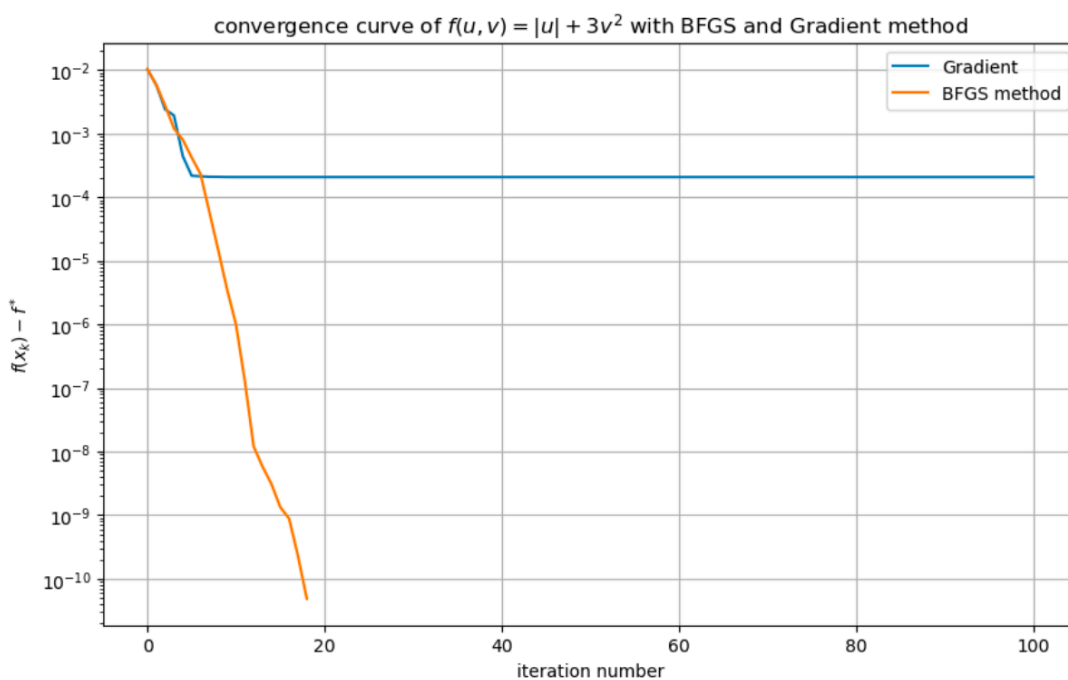
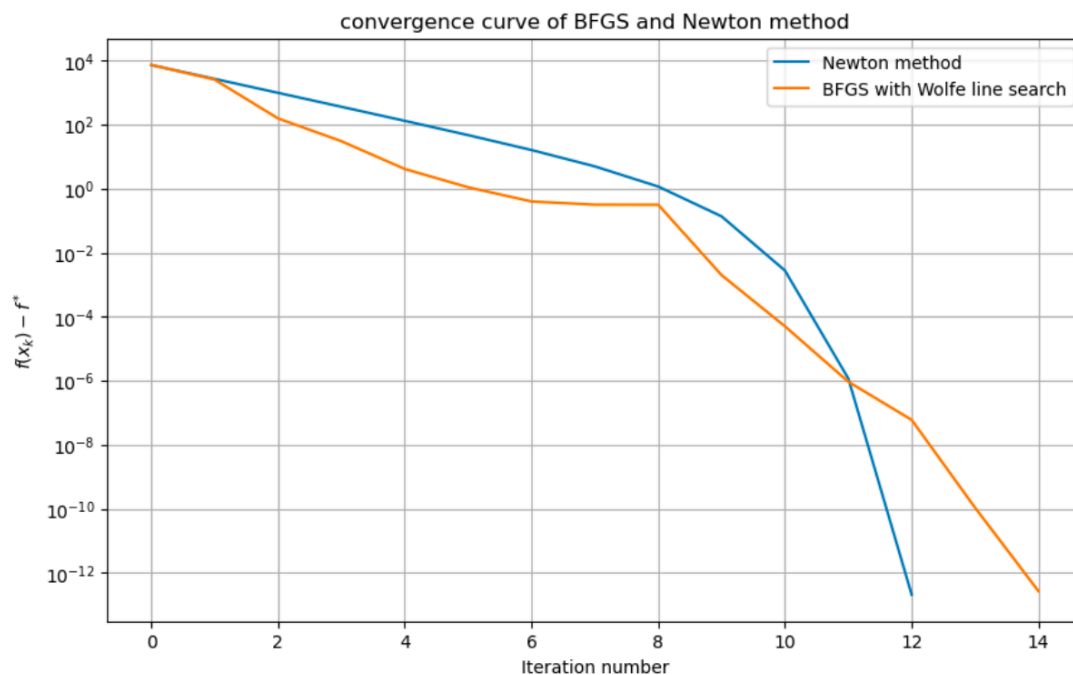
1 Initialization: Choose  $0 < c_1 < c_2 < 1$ , and set  $t = 1$ ,  $\alpha = 0$ ,  $\beta = +\infty$ ;
2 while True do
3   if  $f(x + td) > f(x) + c_1 t \nabla f(x)^T d$  then
4     | set  $\beta = t$  and reset  $t = \frac{1}{2}(\alpha + \beta)$  ;
5   else if  $\nabla f(x + td)^T d < c_2 \nabla f(x)^T d$  then
6     | set  $\alpha = t$ , and reset  $t = \begin{cases} 2\alpha, & \text{if } \beta = +\infty \\ \frac{1}{2}(\alpha + \beta), & \text{Otherwise} \end{cases}$ 
7   else
8     | break ;
9   end if
10 end while

```

- (a) 编程实现采用 Wolfe 线搜索的 BFGS 方法，利用其求解 3.2 (b) 的问题，并与牛顿法进行比较。
- (b) 考虑非平滑凸问题 $f(u, v) = |u| + 3v^2$ ，利用梯度下降法与 BFGS 方法，采用 Wolfe 线搜索进行求解。如果随机生成 $u_0 \neq 0$ 和 $v_0 \neq 0$ 的初始点，两个方法收敛吗？（注意：迭代过程将永远无法触及 $u_i = 0$ 区域，所以各个点都局部可微的）
画出 $f(x_k) - f^*$ 与迭代次数 k 直接的关系，并解释结果。
- (c) 考虑如下问题 $f(u, v) = |u|^3 + v + \frac{1}{2}v^2$ ，编程实现梯度下降法、近端梯度法、牛顿法、拟牛顿法求解，要求与上题一致。

4.1 solution

(a)



(b)

梯度下降法不收敛，拟牛顿法收敛。当梯度下降法迭代几步后，Armijo 条件难以满足导致 Wolfe 条件找不到合适步长，会使得步长趋近于 0, 收敛很慢。

(c)

我们首先计算近端梯度算子:

$$\begin{aligned}\text{prox}_{th}(x) &= \arg \min_u \frac{1}{2} \|u - x\|_2^2 + t|u_{[1]}|^3 \\ &= \arg \min_u \frac{1}{2} (u_{[1]} - x_{[1]})^2 + \frac{1}{2} (u_{[2]} - x_{[2]})^2 + t|u_{[1]}|^3\end{aligned}$$

这里 $u_{[1]}$ 表示 u 向量第一维。

显然 $u_{[2]} = x_{[2]}$ 。

接下来考虑问题

$$\arg \min_{u_{[1]}} \frac{1}{2} (u_{[1]} - x_{[1]})^2 + t|u_{[1]}|^3$$

由于对称性, 我们考虑当 $x_{[1]} \geq 0$ 时最优解一定 $u_{[1]} \geq 0$, 这是因为 $u_{[1]}$ 对于绝对值相等的一正一负取值, 取正时目标函数都更小。

计算梯度为 0, 我们得到

$$u_{[1]} = \frac{-1 + \sqrt{1 + 12tx_{[1]}}}{6t}$$

同样当 $x_{[1]} \leq 0$ 时最优解一定 $u_{[1]} \leq 0$, 计算梯度为 0, 我们得到

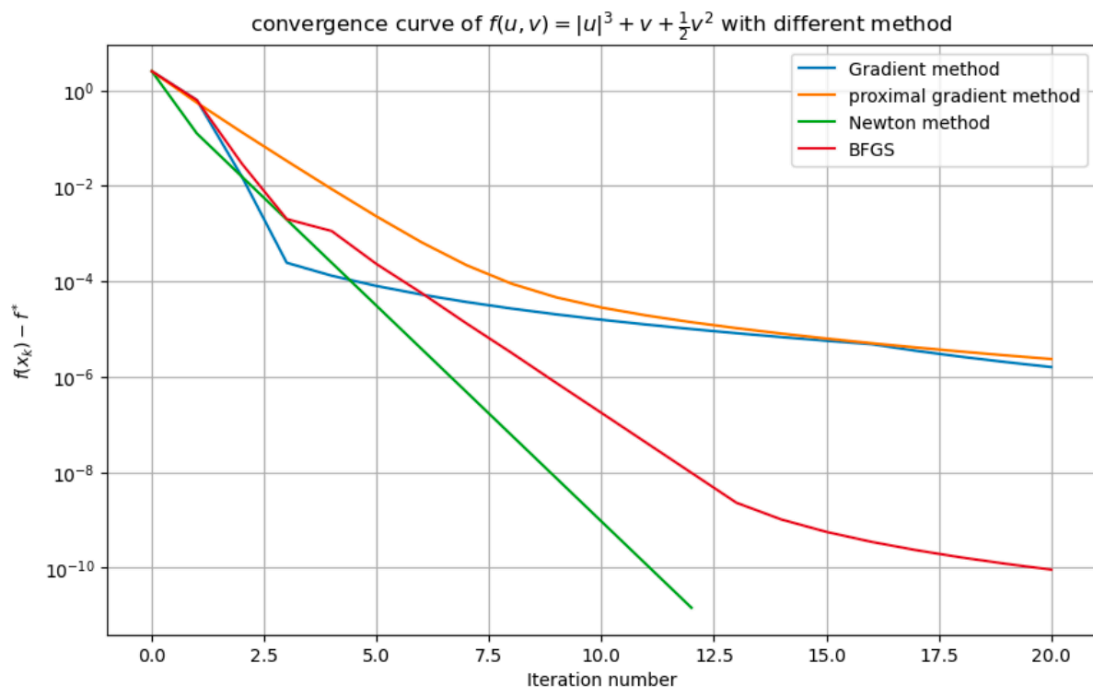
$$u_{[1]} = \frac{1 - \sqrt{1 - 12tx_{[1]}}}{6t}$$

综上有

$$\text{prox}_{th}(x) = \left(\text{sign}(x_{[1]}) \frac{-1 + \sqrt{1 + 12t|x_{[1]}|}}{6t}, x_{[2]} \right)^T$$

那么

$$\begin{aligned}x^{k+1} &= \text{prox}_{t_k h}(x^k - t_k \nabla g(x^k)) = \text{prox}_{t_k h} \left(\begin{pmatrix} x_{[1]}^k \\ x_{[2]}^k - t_k(1 + x_{[2]}^k) \end{pmatrix} \right) \\ &= \begin{pmatrix} \text{sign}(x_{[1]}^k) \frac{-1 + \sqrt{1 + 12t_k|x_{[1]}^k|}}{6t_k} \\ x_{[2]}^k - t_k(1 + x_{[2]}^k) \end{pmatrix}\end{aligned}$$



4.2 比较一阶算法、二阶算法、与拟牛顿法的优劣。

4.2 solution

Table 2: 一阶算法、二阶算法与拟牛顿法对比

特性	一阶算法（梯度下降）	二阶算法（牛顿法）	拟牛顿法（BFGS/L-BFGS）
内存复杂度	$O(n)$	$O(n^2)$	$O(n^2)$ (BFGS) / $O(mn)$ (L-BFGS, $m \ll n$)
计算复杂度	$O(n)$	$O(n^3)$ (Hessian 求逆)	$O(n^2)$ (BFGS) / $O(mn)$ (L-BFGS)
收敛速度	线性收敛	二次收敛	超线性收敛
条件数敏感性	高（病态收敛极慢）	无（尺度不变）	低（近似牛顿尺度矫正）
鲁棒性	强（抗梯度噪声）	弱 (Hessian 病态/非正定易失败)	中等
适用规模	大规模	中小规模	中大规模 (L-BFGS 首选)

核心说明

- **一阶算法**：仅用梯度，简单低成本，但病态问题收敛极慢。
- **二阶算法（牛顿法）**：利用 Hessian，二次收敛、不受条件数影响，但求逆代价高、数值不稳定。
- **拟牛顿法**：用梯度迭代更新 Hessian 逆近似，兼具超线性收敛与低代价；**L-BFGS** 通过保留最近 m 步信息，将内存/计算降为 $O(mn)$ ，是大规模优化首选。